Knowledge Graph Report

1. Introduction

The knowledge graph implemented belongs to the arts and cultural domain, particularly museum art exhibits. The main pipeline can be executed by running python main.py and the ontology gaps can be resolved by running python rag_system.py.

Github Repository: https://github.com/katiepreed/KE-COURSEWORK-2

Team members: Ekaterina Polyakova (K21008618), Jasia Fatima (K24031151), Eva Modwel (K24047844), Mariam Hafiz (K23115695), Zainab Alsaleh (K23073170), and Lupupa Chansa (K23006355).

2. Data Source Selection

The structured data source used is the Metropolitan Museum of Art Collection API, which provides access to metadata for over 490 000 artworks in structured JSON responses. Data ingestion is handled by load_data.py. The API is queried using keywords chosen to cover the object types and thematic categories in the ontology. Results are stored in a single data.json file so that the knowledge graph is reproducible.

The unstructured data is textual content sourced from an article on The Collector (25 Famous Paintings). It was selected for its narrative style describing artworks and museum exhibits, as well as its clearly formatted HTML tags which facilitate targeted extraction of entities and relations.

3. Extension of existing ontologies

Schema.org and CIDOC CRM were selected as existing ontologies, being widely adopted standards for general-purpose and cultural heritage data respectively.

Classes adopted from CIDOC CRM are E22_Human_Made_Object, E39_Actor, E21_Person, and E53_Place. Classes adopted from Schema.org are Painting, Sculpture, Museum, and Person. The equivalentClass axiom bridges the classes, where schema:Person is equivalent to crm:E21_Person, and Painting and Sculpture are declared equivalent to their Schema.org counterparts while also being subclasses of E22_Human_Made_Object.

Custom subclasses extend the hierarchy: E22_Human_Made_Object has Vase, Ceramic, Jewellery, and Scroll; Sculpture has Statue and Figurine; E21_Person has Artist; and E53_Place has Country, Region, and City. A Theme class was introduced with four subclasses (NatureTheme, AnimalTheme, ReligiousTheme, MythologicalTheme), along with a Department class for museum divisions.

Properties adopted from CIDOC CRM include P55_has_current_location, P102_has_title, and P108i_was_produced_by. From Schema.org: dateCreated, name, displayLocation, and locationCreated. The custom createdBy property is a subproperty of both P108i_was_produced_by and schema:creator, with domain E22_Human_Made_Object and range Artist. Additional custom object properties include inverse pairs such as displays/displayedBy, hasDepartment/isDepartmentOf, and hasTheme/isThemeOf, alongside data properties startDate, endDate, hasCulture, and mediumDescription.

OWL axioms enforce domain constraints: Painting is disjoint with Sculpture, Vase, Ceramic, Jewellery, and Scroll; bornOn and diedOn are functional; createdBy and displays are asymmetric and irreflexive; and a minimum cardinality of 1 on hasCreated requires every Artist to have created at least one object. Defined classes enable automated inference: Painter and Sculptor are any Artist who has created at least one Painting or Sculpture respectively, and NatureOilPainting is any Painting with a NatureTheme and mediumDescription "oil on canvas".

4. Mappings

4.1 Structured Data Source Data Prep

During data ingestion and conversion, objects lacking a title are discarded since the title is required for URI generation. Duplicate object IDs are filtered at the triple-conversion stage and "unidentified" artist records are excluded from the artist class. URI identifiers for entities are generated via the make_uri() function in uri_utils.py. This function normalises unicode characters, strips possessive forms and parenthetical content, removes special characters, then capitalises each word and joins them with underscores. Before this, a lookup against an ALIASES dictionary normalises common variants so that semantically identical entities resolve to the same URI. For artworks, the object_id is appended to ensure uniqueness across objects with the same title.

4.2 Structured Data Source Mapping

The mapping from MET API data to RDF triples is implemented in structured_data_mapping.py. The classify_object() function determines the ontology class of each artwork by concatenating its metadata fields into a single lowercase string and scanning it against the OBJECT_TYPE_MAP dictionary. For example, if "porcelain" is detected in any of these fields, the artwork is assigned the class myont:Ceramic. The remaining JSON fields are mapped to RDF properties. Scalar metadata are converted to literals, while fields denoting entities are instantiated as URI-identified individuals. A set of dictionaries tracks created instances, ensuring each entity is created only once. The assign_themes() function then performs a keyword scan over different fields against the THEME_MAP dictionary to annotate artworks with thematic categories.

4.3 Unstructured Data Source Data Prep

HTML content is retrieved using the requests library and parsed using BeautifulSoup, targeting <p> and <figcaption> tags. Text is cleaned using regular expressions to remove whitespace and formatting inconsistencies, then processed using spaCy for sentence segmentation and named entity recognition (NER). NER identifies relevant entities: persons, works of art, organisations, locations, and dates. Only sentences containing at least one relevant entity type are retained, reducing noise and focusing extraction on semantically meaningful content.

4.4 Unstructured Data Source Mapping

Relation extraction is performed using the REBEL model (Babelscape/rebel-large), which generates structured [subject, predicate, object] triples from text. Extracted relations are mapped to ontology predicates via the RELATION_MAP dictionary. In addition, entities identified by spaCy's NER component are assigned to an ontology class via the ENTITY_MAP dictionary. Entity typing uses a simple lookup but may introduce inaccuracies where labels do not precisely align with ontology classes.

Triples are only retained if their subject or object matches a spaCy-recognized entity, reducing hallucinated relations. Known limitations include spaCy classification errors on art-domain entities and REBEL occasionally producing incomplete or ambiguous relations. REBEL was selected over FLAN-T5 as it is more specialised for relation extraction and produced stronger results for this pipeline.

5. Queries

Twenty competency questions were used to evaluate the ontology across different levels of complexity, ranging from single-pattern lookups to multi-join queries that combine several properties and classes. Some queries use aggregation, transitive reasoning over locatedIn property, and regex-based filtering on literal values. The list of questions and their corresponding SPARQL queries are available in cq_sparql.txt. The first 10 CQ were derived from discussing as a team with modelling experts and the last 10 CQ were generated using an LLM. A justification of prompts used is available in the prompt report.

6. Evaluation Methodology

The final knowledge graph, after the RAG completion, was evaluated using eval.py, which relies on owlready2. Since owlready2 does not support Turtle directly, the ontology is parsed from .ttl and serialised as OWL/XML before evaluation.

Consistency checking via the HermiT reasoner confirmed the ontology is logically consistent, with 35 classes, 33 properties, and 838 individuals. Each class has at least one instance, with crm:E22_Human_Made_Object being the most instantiated superclass (348 individuals). Reasoning capability was measured by applying OWL RL deductive closure via owlrl, expanding the ontology from 8,121 to 18,209 triples. All 20 SPARQL queries returned results successfully via rdflib, achieving 100% CQ coverage with negligible execution times. Pipeline performance was measured using psutil: the OWL file is 719.5 KB, loading took 0.15s using 0.20 MB of additional memory, serialising back to Turtle took 0.09s, and the overall script footprint was 88.73 MB.

SPARQL answers were compared against LLM responses for all 20 CQs. SPARQL answers were deterministic, reproducible, and grounded in structured data, ensuring high precision but limited by KG completeness. LLM responses were more flexible but introduced inaccuracies and lacked consistency across runs. For CQ1 ("Which French artists have lived for more than 70 years?"), SPARQL returned an exact list with birth and death years, while the LLM produced a summarised answer that omitted or grouped details. This highlights the core trade-off that the KG provides trustworthy but potentially incomplete answers, while the LLM offers broader but less reliable outputs.